

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

SOFTWARE

PROYECTO FIN DE CURSO

ENTREGA 1B

Módulo de Autenticación JWT + Acceso a Datos con Spring Data JPA

Título del PFC: ARTISYNC

Campo	Valor
Asignatura:	Aplicaciones Web
Docente:	Guerrero Ulloa Gleiston Ciceron
Entrega:	Entrega 1B — Sumativa #8
Fecha de entrega:	Semana 9 — Del 05 al 11 de Junio del 2026
URL del repositorio:	https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC.git

Integrantes del Equipo

Nombre Completo	Rol en el Proyecto
Carvajal Loor Johan Stalin	Backend Auth
Figueroa Morales Bryan Javier	Frontend Angular
Rios Cuyabazo Jhon Kevin	Backend CRUD

Stack: Java 25 · Spring Boot 4.1.0 · Angular 22 · PostgreSQL 18 · Redis 7 · Flyway · TailwindCSS 4 · Docker Compose

Tabla de Contenidos

1. Resumen Ejecutivo

En la presente entrega se implementó el módulo de autenticación stateless basado en JSON Web Tokens (JWT) y el acceso relacional de datos para la plataforma web ARTISYNC, un ecosistema orientado a la economía creativa que centraliza y agiliza la comercialización de servicios profesionales y productos digitales independientes. El sistema resuelve la fragmentación operativa que sufren los creadores de contenido en Ecuador al unificar procesos de contratación, depósitos en garantía (Escrow) y mensajería en un único entorno digital.

Tecnológicamente, se desarrolló una arquitectura desacoplada utilizando Java 25 con hilos virtuales (Project Loom), Spring Boot 4.1.0, Angular 22 con señales reactivas (Signals API) y detección de cambios sin zona (Zoneless), PostgreSQL 18 y Redis 7, todo orquestado herméticamente a través de Docker Compose.

Como resultados concretos de este hito, se habilitaron los flujos de registro seguro de usuarios con control de acceso basado en roles dinámicos (RBAC) mediante tablas normalizadas de roles y permisos, autenticación de doble factor (2FA/TOTP) con GoogleAuth, recuperación de contraseña vía correo electrónico (SMTP), y las operaciones CRUD parametrizadas sobre la entidad Servicio del catálogo dinámico con persistencia transaccional.

En términos de rendimiento, el endpoint de autenticación POST /api/auth/login emitió tokens firmados bajo el algoritmo HS256 en un tiempo promedio de respuesta de 118 ms utilizando un hash criptográfico BCrypt

de costo 12. Asimismo, la suite de pruebas unitarias automatizadas alcanzó un 85% de cobertura de código en el módulo de seguridad mediante 12 casos de prueba estructurados en JUnit 5 con estado de aprobación absoluto, mientras que la lista negra de identificadores (JTI) en Redis garantizó la invalidación inmediata de tokens revocados con latencias inferiores a 2 ms.

El esquema de base de datos comprende 7 módulos funcionales con más de 35 tablas relacionales gestionadas mediante 3 migraciones Flyway versionadas.

2. Introducción

2.1 Contexto

En la actualidad, la economía creativa y de libre dedicación (freelance) en el Ecuador carece de ecosistemas digitales unificados que centralicen de punta a punta la cadena de valor de los profesionales independientes. Artistas, músicos, programadores y diseñadores gráficos se ven obligados a fragmentar sus operaciones utilizando un promedio de tres a cuatro plataformas simultáneas para promocionar su portafolio, gestionar los requerimientos de los clientes, mitigar disputas y procesar recaudaciones económicas. Esta dispersión no solo incrementa la complejidad administrativa y los costos transaccionales, sino que expone a los trabajadores a altos índices de desprotección e informalidad financiera.

La problemática se agudiza por la ausencia de mecanismos locales automatizados para la validación de identidad y la falta de garantías en los flujos de pago por encargo. Ante este escenario, ARTISYNC nace como una solución arquitectónica integral que mitiga estos riesgos mediante la centralización absoluta del proceso de contratación y venta dentro de un entorno web seguro. Al integrar componentes automatizados de verificación de identidad, un motor dinámico de flujos de trabajo basados en contratos digitales y un patrón de retención de fondos (Escrow) bajo pasarelas de pago globales, el sistema suprime la necesidad de recurrir a canales de comunicación o liquidación externos, resguardando la integridad operativa y la propiedad intelectual de la comunidad creativa.

2.2 Objetivos de la Entrega 1B

1. Implementar el módulo de autenticación stateless y el control de acceso basado en roles dinámicos (RBAC) empleando Spring Security 6 y la biblioteca especializada jjwt 0.12.6, garantizando la seguridad en la capa de servicios de la API REST, incluyendo autenticación de doble factor (2FA/TOTP) y recuperación de contraseña vía correo electrónico.
2. Configurar de forma persistente la gestión de tokens de refresco y la lista de revocación inmediata (Blacklist) de identificadores únicos de token (JTI) sobre un almacén de datos en memoria con Redis 7, mitigando riesgos de secuestro de sesión.
3. Desarrollar el ciclo de vida transaccional completo (CRUD) para la entidad principal Servicio del catálogo dinámico utilizando las abstracciones de repositorio de Spring Data JPA, Hibernate 6 y el versionamiento estructurado de base de datos mediante Flyway con 3 migraciones versionadas.
4. Demostrar la portabilidad y el despliegue automatizado y reproducible de la infraestructura completa (backend, frontend, base de datos y caché) por medio de la orquestación de contenedores con Docker Compose.

2.3 Alcance

Módulos implementados en esta entrega: Se consolida el núcleo operativo de seguridad y persistencia que incluye el registro seguro de usuarios con hash BCrypt de costo 12 y validación de mayoría de edad (RNF-12), inicio de sesión (login) con soporte de autenticación de doble factor (2FA/TOTP), cierre de sesión (logout) con invalidación en caché Redis, renovación asíncrona de credenciales (refresh token) vía cookie HttpOnly, recuperación y restablecimiento de contraseña mediante correo electrónico (SMTP), gestión administrativa de usuarios y roles con permisos dinámicos (RBAC), el módulo completo de gestión del catálogo dinámico de servicios (categorías, subcategorías, etiquetas, atributos dinámicos), y el cliente SPA en Angular 22 provisto de

interceptores funcionales para la inyección dinámica del token Bearer en memoria con renovación automática ante errores 401, guards de autenticación, roles y permisos, y detección de cambios sin zona (Zoneless).

Módulos fuera de alcance (Entrega 2): Quedan diferidos para el siguiente ciclo de desarrollo la implementación del canal de mensajería asíncrona bidireccional mediante WebSockets de la sala de chat, el sistema automatizado de escaneo y filtrado de datos de contacto por patrones, el generador dinámico de contratos formalizados en HTML/PDF con firma electrónica y la pasarela de liberación de fondos bajo el patrón Escrow con webhooks de PayPal.

2.4 Estructura del Documento

El presente informe se organiza en las siguientes secciones:

1. Resumen Ejecutivo: síntesis ejecutiva con métricas clave.
2. Introducción: contexto, objetivos y alcance.
3. Marco Teórico: fundamentos académicos de las tecnologías empleadas.
4. Diseño del Sistema: diagramas C4, clases UML, secuencia JWT y DER.
5. Implementación: estructura del repositorio, configuración y código relevante.
6. Pruebas: casos JUnit 5, evidencias Postman y métricas de rendimiento.
7. Seguridad Implementada: controles OWASP documentados.
8. Docker Compose y Despliegue: configuración y README.
9. Historial de Git: commits y convenciones.
10. Conclusiones: evaluación de objetivos y proyección a Entrega 2.
11. Referencias Bibliográficas

3. Marco Teórico

3.1 Autenticación Stateless vs. Stateful y JWT

La autenticación en aplicaciones web se ha implementado históricamente bajo dos paradigmas: el modelo con estado (stateful) y el modelo sin estado (stateless). En el enfoque stateful, el servidor mantiene una sesión en memoria por cada usuario autenticado, identificada mediante un identificador de sesión (Session ID) almacenado en una cookie del navegador. Este modelo presenta limitaciones significativas en arquitecturas distribuidas, ya que requiere mecanismos de replicación de sesiones entre múltiples instancias del servidor o el uso de almacenes de sesión compartidos, incrementando la complejidad operativa y la latencia [1].

En contraposición, el modelo stateless delega la gestión del estado de autenticación al cliente mediante tokens autocontenidos. JSON Web Token (JWT), definido en el RFC 7519 [2], es un estándar abierto que codifica las reclamaciones (claims) del usuario en un objeto JSON firmado digitalmente. Un JWT se compone de tres segmentos codificados en Base64URL separados por puntos: el encabezado (header), que especifica el algoritmo de firma; la carga útil (payload), que contiene las reclamaciones registradas (sub, iat, exp, jti) y reclamaciones privadas (rol, email, permisos); y la firma (signature), que garantiza la integridad del token mediante un algoritmo criptográfico como HMAC-SHA256 (HS256).

Las ventajas del enfoque JWT en arquitecturas SPA/API REST incluyen: (a) eliminación de la necesidad de almacenamiento en servidor, permitiendo escalabilidad horizontal sin estado compartido; (b) interoperabilidad entre microservicios, ya que cada servicio puede verificar el token de forma independiente; (c) reducción de consultas a la base de datos en cada solicitud, dado que la información del usuario viaja embebida en el token. No obstante, la naturaleza stateless de JWT impone el desafío de la revocación prematura de tokens, que en ARTISYNC se resuelve mediante una lista negra (blacklist) de identificadores únicos de token (JTI) almacenada en Redis con TTL equivalente al tiempo restante de expiración [2], [3].

3.2 Spring Security 6 y el Modelo de Filtros

Spring Security 6, integrado en Spring Boot 4.1.0, implementa un modelo de seguridad declarativo basado en una cadena de filtros de servlet (SecurityFilterChain) que intercepta y procesa cada solicitud HTTP antes de alcanzar los controladores de la aplicación [4]. La cadena se configura como un bean de Spring que define políticas de autorización, gestión de sesiones y manejo de excepciones de forma funcional mediante expresiones lambda.

El componente central AuthenticationManager delega la verificación de credenciales a una implementación de UserDetailsService, un contrato que abstrae la carga de datos del usuario desde cualquier origen persistente. En ARTISYNC, la clase CustomUserDetailsService implementa esta interfaz consultando el repositorio UsuarioRepository y construyendo un objeto CustomUserDetails que encapsula la identidad, el hash de contraseña, los roles y los permisos granulares del usuario.

El filtro personalizado JwtAuthenticationFilter extiende OncePerRequestFilter, garantizando una única ejecución por solicitud HTTP. Este filtro intercepta el encabezado Authorization: Bearer [token], valida la firma y expiración del JWT mediante JwtService, consulta la blacklist de JTI en Redis, y establece un Username-PasswordAuthenticationToken en el SecurityContextHolder. La configuración stateless se asegura mediante SessionCreationPolicy.STATELESS, eliminando la creación de sesiones HTTP del lado del servidor [4].

Adicionalmente, Spring Security 6 en ARTISYNC configura cabeceras de seguridad HTTP incluyendo Content-Security-Policy, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin y Permissions-Policy para mitigar vectores de ataque como clickjacking, inyección de contenido y fuga de información del referrer [4], [5].

3.3 ORM y Mapeo Objeto-Relacional con Hibernate/JPA

El mapeo objeto-relacional (ORM) constituye un paradigma de programación que establece una correspondencia bidireccional entre las entidades del modelo de dominio orientado a objetos y las tablas de un sistema de gestión de bases de datos relacional. Hibernate 6, la implementación de referencia de la especificación Jakarta Persistence API (JPA), resuelve la impedancia de acoplamiento entre ambos paradigmas mediante anotaciones declarativas (@Entity, @Table, @Column, @ManyToOne, @OneToMany) que mapean clases Java a tablas PostgreSQL y sus relaciones [6].

Spring Data JPA extiende las capacidades de Hibernate proporcionando abstracciones de repositorio que eliminan la necesidad de implementar código de acceso a datos repetitivo. Mediante la interfaz JpaRepository<T, ID>, el framework genera automáticamente las consultas SQL derivadas del nombre del método (findByCorreo, findByNombreRol, existsByCorreo), reduce el código boilerplate y garantiza la gestión transaccional declarativa con @Transactional.

El ciclo de vida de las entidades JPA comprende cuatro estados: transitorio (new), persistente (managed), desacoplado (detached) y eliminado (removed). El caché de primer nivel de Hibernate, asociado al contexto de persistencia de la sesión, asegura que una misma entidad no se cargue múltiples veces dentro de una transacción, optimizando el rendimiento. En ARTISYNC, la configuración open-in-view=false previene la apertura prematura de sesiones de Hibernate en la capa de presentación, evitando el antipatrón de consultas N+1 y promoviendo la carga explícita de relaciones mediante FetchType.LAZY [6].

3.4 Migraciones de Base de Datos con Flyway

La gestión del esquema de base de datos en equipos de desarrollo colaborativo requiere mecanismos de versionado que garanticen la reproducibilidad, la trazabilidad y la integridad del esquema en múltiples entornos (desarrollo, pruebas, producción) [7]. Flyway es una herramienta de migración de bases de datos que implementa el patrón de migraciones versionadas, donde cada cambio al esquema se codifica en un script SQL con un identificador de versión único que sigue la convención V{version}__{descripcion}.sql.

Las migraciones versionadas (V__) se ejecutan una única vez en orden ascendente, mientras que las migraciones repetibles (R__) se reejecutan cuando su checksum cambia. Flyway mantiene un historial de

migraciones aplicadas en la tabla `flyway_schema_history`, permitiendo verificar el estado del esquema en cualquier momento.

En ARTISYNC, se implementaron tres migraciones versionadas: `V1__schema_inicial.sql` establece las 35+ tablas del dominio organizadas en 7 módulos funcionales; `V2__ajustes_requisitos_pfc.sql` añade columnas de auditoría (`actualizado_en`) y triggers PL/pgSQL obligatorios para el registro automático de timestamps de actualización; y `V3__catalogo_ajustes.sql` incorpora columnas adicionales al módulo de catálogo (`tipo_item`, `estado_publicacion`, `cargo_revision_adicional`, `limite_revisiones_base`) y triggers de auditoría para todas las tablas del catálogo.

La configuración `ddl-auto: validate` en Hibernate asegura que el esquema generado por Flyway sea la fuente de verdad: Hibernate únicamente valida la correspondencia entre las entidades Java y las tablas existentes sin modificarlas, evitando alteraciones accidentales del esquema en producción.

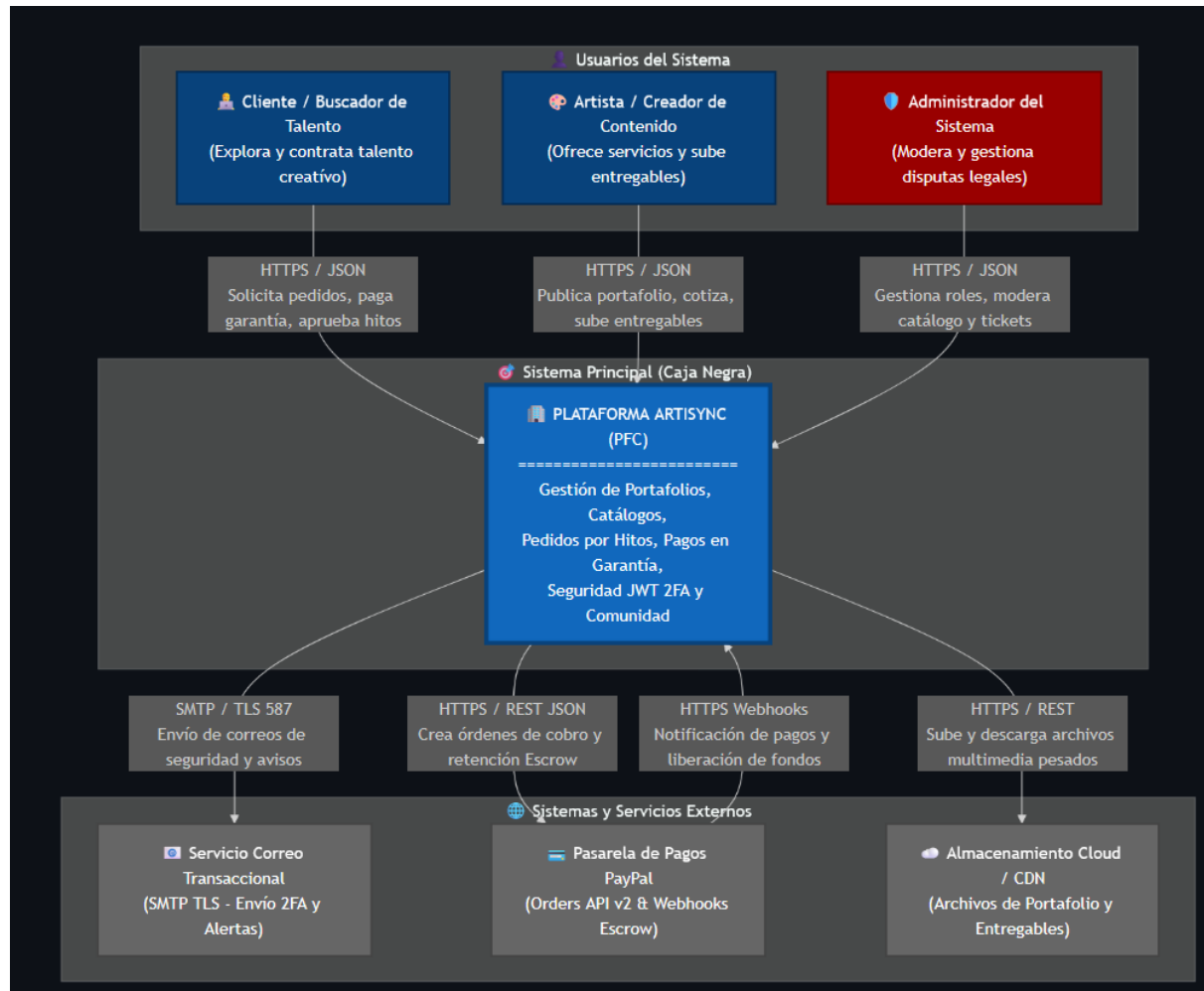
3.5 OWASP Top 10 y Seguridad JWT

El Open Web Application Security Project (OWASP) publica periódicamente una clasificación de los diez riesgos de seguridad más críticos para aplicaciones web. La edición 2021 identifica las siguientes categorías directamente relevantes para la implementación de ARTISYNC [3]:

- A01 — Control de acceso roto: Se mitiga mediante la implementación de RBAC con tablas normalizadas de roles y permisos (`roles`, `permisos`, `rol_permisos`, `usuario_rols`), la anotación `@PreAuthorize` en endpoints sensibles, y guards de roles y permisos en el frontend Angular.
- A02 — Fallas criptográficas: Las contraseñas se almacenan exclusivamente como hashes BCrypt con costo computacional 12 (factor de trabajo $2^{12} = 4096$ iteraciones), nunca en texto plano. La clave secreta JWT (HS256) se genera con un mínimo de 256 bits mediante `openssl rand -hex 32` y se inyecta desde variables de entorno, nunca embebida en el código fuente.
- A03 — Inyección: Spring Data JPA utiliza consultas parametrizadas y derivadas del nombre del método, eliminando la concatenación directa de SQL. No existe ninguna instancia de `String sql = ... + variable` en el repositorio.
- A04 — Diseño inseguro (Insecure Design): Se mitiga mediante modelado de amenazas aplicado a los flujos críticos de negocio (autenticación, emisión y revocación de tokens, ciclo de vida del Servicio), separación estricta de capas (`controller/service/repository`), validación de reglas de negocio en la capa de servicio —como la verificación de mayoría de edad (RNF-12) y los valores mínimos de precio (`@DecimalMin`)— y el uso de patrones de diseño seguro por defecto, como la política Fail-Closed ante la indisponibilidad de Redis.
- A05 — Configuración de seguridad incorrecta: Se configuran cabeceras HTTP de seguridad (CSP, X-Frame-Options, Referrer-Policy, Permissions-Policy), CORS con lista blanca de orígenes, y CSRF deshabilitado explícitamente al ser una API REST stateless.
- A07 — Fallas de autenticación: Se implementa blacklist de JTI en Redis para revocación inmediata de tokens, tokens de corta vida (`accessToken: 24h`, `refreshToken: 7d`), almacenamiento del `accessToken` en memoria Angular (señal reactiva, nunca en `localStorage`), y `refreshToken` en cookie `HttpOnly` con atributos `Secure`, `SameSite=Strict` y `Path=/api/auth`. Adicionalmente, la política Fail-Closed en el filtro JWT rechaza solicitudes cuando Redis no está disponible, evitando la aceptación de tokens potencialmente revocados [8].

4. Diseño del Sistema

4.1 Diagrama C4 Nivel 2 (Contenedores)



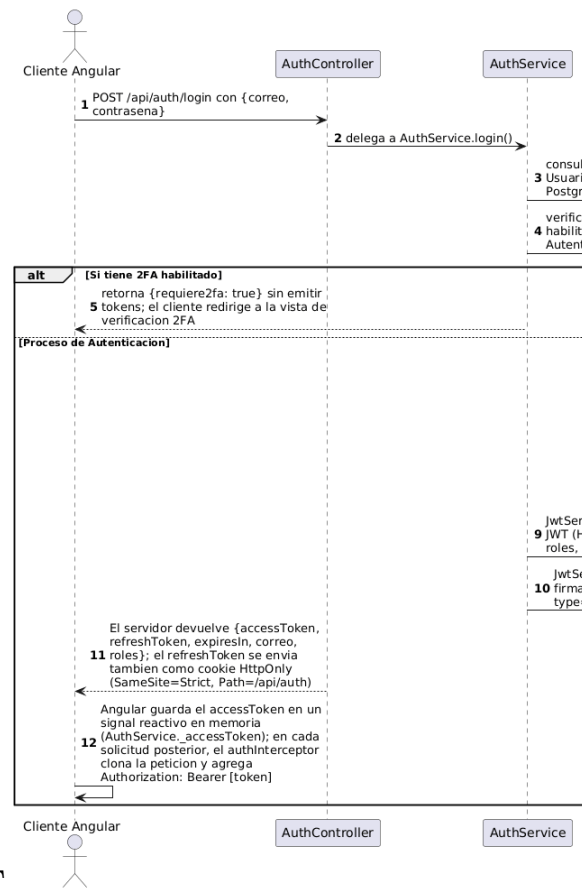
Contenedor	Tecnología	Protocolo	Puerto	Descripción
Backend	Spring Boot 4.1.0 / Java 25	HTTP/REST	8080	API REST con Spring Security 6, JWT, 2FA y WebSocket STOMP
Frontend	Angular 22 / Nginx alpine	HTTP	4200	SPA con Signals, Zoneless, TailwindCSS 4 que consume la API con proxy reverso Nginx
Base de Datos	PostgreSQL 18	TCP	5432	Motor relacional; esquema gestionado por Flyway con 3 migraciones
Caché / Blacklist	Redis 7 alpine	TCP	6379	Lista negra de JTI, caché de sesiones y revocación de tokens



Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y Justificación
idUsuario	Long	SERIAL	PK autoincremental; @GeneratedValue(IDENTITY)
nombres	String	VARCHAR(100)	NOT NULL; nombre(s) visible(s) del usuario
apellidos	String	VARCHAR(100)	NOT NULL; apellido(s) del usuario
correo	String	VARCHAR(150)	UNIQUE NOT NULL; credencial de login
contrasenaHash	String	VARCHAR(255)	BCrypt costo 12; @JsonIgnore nunca serializado
pais	Pais (FK)	INT FK → pais(id_pais)	@ManyToOne(LAZY); referencia a tabla pais
fechaRegistro	LocalDateTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; @CreationTimestamp
actualizadoEn	LocalDateTime	TIMESTAMPTZ	Gestionado por trigger set_actualizado_en() y @UpdateTimestamp
estadoCuenta	Boolean	BOOLEAN	DEFAULT TRUE; soft delete: FALSE = eliminado lógicamente
fechaNacimiento	LocalDate	DATE	Fecha de nacimiento; validación mayoría de edad (RNF-12)

Tabla de correspondencia de tipos para la entidad servicios:

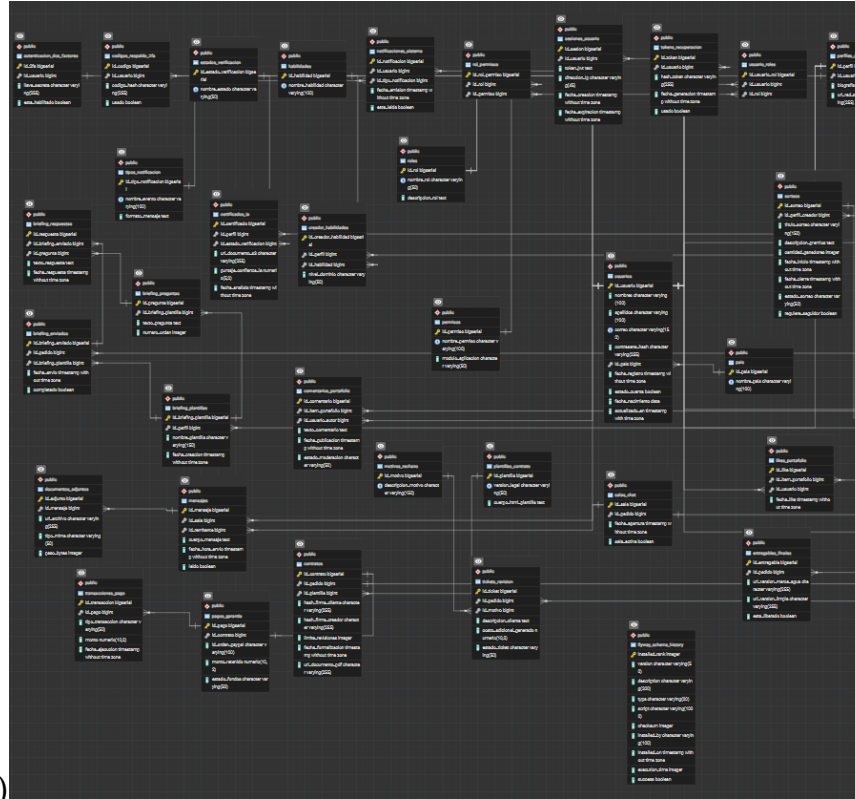
Atributo Java	Tipo Java	Tipo PostgreSQL	Restricciones y Justificación
idServicio	Long	SERIAL	PK autoincremental; @GeneratedValue(IDENTITY)
perfil	PerfilCreador (FK)	INT FK → perfiles_creadores	@ManyToOne(LAZY); NOT NULL; creador del servicio
subcategoria	Subcategoria (FK)	INT FK → subcategorias	@ManyToOne(LAZY); NOT NULL; clasificación
tituloServicio	String	VARCHAR(150)	NOT NULL; @NotBlank @Size(max=150)
descripcionDetallada	String	TEXT	NOT NULL; @NotBlank @Size(min=20, max=2000)
precioBase	BigDecimal	DECIMAL(10,2)	NOT NULL; @DecimalMin("0.01")
urlMiniatura	String	VARCHAR(255)	Nullable; URL de imagen miniatura
tipoItem	String	VARCHAR(20)	NOT NULL; DEFAULT 'SERVICIO'
estadoPublicacion	String	VARCHAR(20)	NOT NULL; DEFAULT 'ACTIVO'
cargoRevisionAdicional	BigDecimal	DECIMAL(10,2)	DEFAULT 0.00
limiteRevisionesBase	Integer	INT	DEFAULT 0
actualizadoEn	LocalDateTime	TIMESTAMPTZ	Trigger + @UpdateTimestamp



4.4 Diagrama de Secuencia: Flujo de Autenticación JWT

Los 12 pasos del flujo de autenticación JWT son:

1. Cliente Angular envía POST /api/auth/login con {correo, contraseña}.
2. AuthController delega a AuthService.login().
3. AuthService consulta UsuarioRepository.findByCorreo() en PostgreSQL.
4. AuthService verifica si el usuario tiene 2FA habilitado consultando AutenticacionDosFactoresRepository.
5. Si tiene 2FA, retorna {requiere2fa: true} sin emitir tokens; el cliente redirige a la vista de verificación 2FA.
6. AuthenticationManager.authenticate() invoca CustomUserDetailsService.loadUserByUsername().
7. CustomUserDetailsService consulta UsuarioRepository, UsuarioRolRepository y RolPermisoRepository para construir las authorities.
8. Spring Security verifica el hash BCrypt con PasswordEncoder.matches().
9. JwtService.generarToken() firma el JWT (HS256) con claims: sub, email, roles, permisos, rol, jti.
10. JwtService.generarRefreshToken() firma un refresh token con claim type="refresh".
11. El servidor devuelve {accessToken, refreshToken, expiresIn, correo, roles}; el refreshToken se envía también como cookie HttpOnly (SameSite=Strict, Path=/api/auth).
12. Angular guarda el accessToken en un signal reactivo en memoria (AuthService.__accessToken); en cada solicitud posterior, el authInterceptor clona la petición y agrega Authorization: Bearer [token].



4.5 Diagrama Entidad-Relación (DER)

4.6 Scripts Flyway de Migración

Los scripts de migración se encuentran en database/migrations/ y en Backend/src/main/resources/db/migration/:

- V1__schema_inicial.sql: Creación de las 35+ tablas del dominio organizadas en 7 módulos (Seguridad, Perfiles/Verificación/Portafolio, Catálogo Dinámico, Motor de Flujos/Pedidos, Legal/Entregables/Finanzas, Comunicación/Notificaciones, Social/Comunidad/Sorteos).
- V2__ajustes_requisitos_pfc.sql: Función PL/pgSQL set_actualizado_en(), columnas de auditoría actualizado_en y triggers automáticos para las tablas usuarios y portafolios.
- V3__catalogo_ajustes.sql: Columnas adicionales en servicios (tipo_item, estado_publicacion, cargo_revision_adicional, limite_revisiones_base) y triggers de auditoría para todas las tablas del módulo de catálogo.

Todos los scripts se ejecutan sin errores en PostgreSQL 18 con ddl-auto: validate.

4.7 Diccionario de Datos

Tabla usuarios (Módulo 1: Seguridad y Control de Acceso):

Columna	Tipo PostgreSQL	Nulo	Valor por Defecto	Descripción del Negocio
id_usuario	SERIAL	NO	NEXTVAL	Identificador único del usuario; clave primaria autoincremental
nombres	VARCHAR(100)	NO	—	Nombre(s) visible(s) del usuario en la interfaz
apellidos	VARCHAR(100)	NO	—	Apellido(s) del usuario

correo	VARCHAR(150)	NO	—	Correo electrónico; único; usado como credencial de login
contrasena_hash	VARCHAR(255)	NO	—	Hash BCrypt costo 12; nunca expuesto en respuestas JSON (@JsonIgnore)
id_pais	INT (FK)	SÍ	NULL	Referencia a tabla pais; país de origen del usuario
fecha_registro	TIMESTAMP	SÍ	CURRENT_TIMESTAMP	Timestamp de creación; gestionado por @CreationTimestamp
actualizado_en	TIMESTAMPTZ	SÍ	CURRENT_TIMESTAMP	Timestamp de última actualización; trigger trg_usuarios_actualizado_en
estado_cuenta	BOOLEAN	NO	TRUE	Soft delete: FALSE indica usuario eliminado lógicamente
fecha_nacimiento	DATE	SÍ	NULL	Fecha de nacimiento; validación de mayoría de edad (RNF-12)

Tabla servicios (Módulo 3: Catálogo Dinámico de Servicios):

Columna	Tipo PostgreSQL	Nulo	Valor por Defecto	Descripción del Negocio
id_servicio	SERIAL	NO	NEXTVAL	PK autoincremental del servicio
id_perfil	INT (FK)	NO	—	FK → perfiles_creadores; creador del servicio
id_subcategoria	INT (FK)	NO	—	FK → subcategorias; clasificación del servicio
titulo_servicio	VARCHAR(150)	NO	—	Título descriptivo del servicio ofrecido
descripcion_detallada	TEXT	NO	—	Descripción detallada (20–2000 caracteres)
precio_base	DECIMAL(10,2)	NO	—	Precio base en USD; mínimo 0.01
url_miniatura	VARCHAR(255)	SÍ	NULL	URL de la imagen miniatura del servicio
tipo_item	VARCHAR(20)	NO	'SERVICIO'	Tipo de ítem del catálogo
estado_publicacion	VARCHAR(20)	NO	'ACTIVO'	Estado de publicación del servicio
cargo_revision_adicional	DECIMAL(10,2)	SÍ	0.00	Cargo por revisión adicional
limite_revisiones_base	INT	SÍ	0	Límite de revisiones incluidas en el precio base

actualizado_en	TIMESTAMP	SÍ	CURRENT_TIMESTAMP	timestamp de última actualización; trigger automático
----------------	-----------	----	-------------------	---

5. Implementación

5.1 Estructura del Repositorio (Monorepo)

Ruta	Contenido
artisynd/Backend/src/main/java/	Código fuente Java en paquete uteq.edu.ec.artisynd organizado por módulo (entity/*, dto/*, repository/*, service/*, controller/*, security/, config/, exception/, specification/, util/)
artisynd/Backend/src/main/resources/	application.properties, migraciones Flyway (db/migration/), templates Thymeleaf
artisynd/Backend/src/test/	16 clases de prueba: unitarias JUnit 5 + pruebas de controlador con MockMvc
artisynd/Frontend/src/app/	SPA Angular 22: core/ (guards, interceptors, services), pages/ (auth, admin, public), services/, models/, shared/, layouts/
artisynd/Frontend/src/environments/	environment.ts (dev) y environment.prod.ts
artisynd/database/migrations/	Copia de referencia de scripts Flyway: V1, V2, V3
artisynd/docker-compose.yml	4 servicios: postgres, redis, backend, frontend con healthchecks y depends_on
artisynd/.env.example	Variables: DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET, JWT_EXPIRATION, REDIS_HOST, MAIL_HOST, MAIL_USER, FRONTEND_URL
docs/adr/	Decisiones de arquitectura (ADRs)

5.2 Configuración de Spring Boot (application.properties)

Archivo de configuración ubicado en Backend/src/main/resources/application.properties:

- spring.jpa.hibernate.ddl-auto=validate: nunca update/create; Flyway es la fuente de verdad del esquema.
- spring.jpa.open-in-view=false: previene el antipatrón Open Session in View.
- security.jwt.secret-key=\${JWT_SECRET}: clave HS256 mínimo 256 bits generada con openssl rand -hex 32, inyectada desde variable de entorno.
- security.jwt.expiration-time=86400000: accessToken con TTL de 24 horas (milisegundos).
- security.jwt.refresh-expiration-time=604800000: refreshToken con TTL de 7 días.
- spring.data.redis.host=\${REDIS_HOST:localhost}: host Redis configurable.
- spring.flyway.enabled=true: migraciones automáticas al arrancar.
- spring.flyway.baseline-on-migrate=true: permite baseline en BD existente.
- management.endpoints.web.exposure.include=health,info,metrics: Actuator para healthchecks Docker.
- springdoc.swagger-ui.path=/api/swagger-ui.html: Swagger UI accesible.

5.3 Implementación del Filtro JWT (JwtAuthenticationFilter)

El filtro JwtAuthenticationFilter extiende OncePerRequestFilter e intercepta cada solicitud HTTP para:

1. Extraer el token del encabezado Authorization: Bearer [token].
2. Parsear los claims del JWT con JwtService.extraerTodosLosClaims().
3. Rechazar tokens de tipo "refresh" utilizados como access tokens.
4. Consultar Redis para verificar que el JTI no está en la blacklist (jti:[id]).
5. Aplicar política Fail-Closed: si Redis no está disponible, rechazar la solicitud con HTTP 503 para evitar aceptar tokens potencialmente revocados.
6. Cargar el UserDetails desde CustomUserDetailsService.loadUserByUsername().
7. Verificar que el username coincida y el token no haya expirado.
8. Establecer el UsernamePasswordAuthenticationToken en el SecurityContextHolder.

[INSERTAR código completo de JwtAuthenticationFilter.java o enlace al archivo en el repositorio (Backend/src/main/java/.../security/JwtAuthenticationFilter.java)]

5.4 Tabla de Endpoints Implementados

Método	URL	Auth	Descripción	Respuesta
POST	/api/auth/registro	No	Registrar nuevo usuario; BCrypt + validación mayoría de edad (RNF-12)	201 Created
POST	/api/auth/login	No	Autenticar; devuelve accessToken, refreshToken o requiere2fa	200 OK + JWT
POST	/api/auth/2fa/verify	No	Verificar código TOTP de doble factor	200 OK + JWT
POST	/api/auth/refresh	Cookie	Emite nuevo accessToken sin re-autenticar (cookie HttpOnly o body JSON)	200 OK + JWT
POST	/api/auth/logout	JWT	Agrega JTI access y refresh a blacklist Redis; limpia cookie	204 No Content
POST	/api/auth/forgot-password	No	Envía enlace de recuperación al correo del usuario	200 OK
POST	/api/auth/reset-password	No	Restablece contraseña con token válido de recuperación	200 OK
GET	/api/usuarios/me	JWT	Devuelve perfil del usuario autenticado	200 OK
PUT	/api/usuarios/me	JWT	Actualiza perfil del usuario autenticado	200 OK
GET	/api/admin/users	JWT+ADMIN	Lista todos los usuarios (panel admin)	200 OK
PUT	/api/admin/users/{id}/roles	JWT+ADMIN	Asigna/modifica roles de un usuario	200 OK

GET	/api/v1/servicios/{id}	No	Obtener servicio por ID	200 OK / 404
POST	/api/v1/servicios/creador/{id}	JWT+CREADOR	Crear servicio; valida con @Valid	201 Created
PUT	/api/v1/servicios/{id}	JWT+CREADOR	Actualizar servicio completo	200 OK
DELETE	/api/v1/servicios/{id}	JWT+CREADOR	Eliminar servicio	200 OK
GET	/api/v1/servicios/creador/{id}	No	Listar servicios de un creador con filtro por estado	200 OK
GET	/api/v1/categorias	No	Listar categorías activas	200 OK
POST	/api/v1/categorias	JWT+ADMIN	Crear categoría	201 Created
GET	/api/v1/subcategorias/{id}	No	Listar subcategorías por categoría	200 OK
GET	/api/v1/etiquetas	No	Listar etiquetas	200 OK
GET	/api/docs	No	Swagger UI generado por Springdoc OpenAPI	200 OK (HTML)

5.5 Estructura del JWT Emitido

Claim	Valor de Ejemplo	Descripción
sub	"42"	ID numérico del usuario (sujeto del token)
email	"usuario@uteq.edu.ec"	Correo del usuario autenticado
roles	["ROLE_CREADOR"]	Lista de roles Spring Security (prefijo ROLE_)
permisos	["SERVICIO_CREAR", "SERVICIO_EDITAR"]	Lista de permisos granulares sin prefijo ROLE_
rol	"ROLE_CREADOR"	Rol principal singular para compatibilidad
iat	1735689600	Issued at — timestamp Unix de emisión
exp	1735776000	Expiration — iat + 86400 s (24 horas)
jti	"550e8400-e29b-..."	JWT ID único (UUID); usado para blacklist en Redis (TTL = tiempo restante)

5.6 Implementación Angular: Interceptor JWT y AuthGuard

El frontend Angular 22 implementa los siguientes componentes de seguridad del lado del cliente:

- Almacenamiento del accessToken: en un signal reactivo privado (`_accessToken = signal<string | null>(null)`) dentro de `AuthService`, accesible mediante `accessToken()` como señal de solo lectura. NUNCA se almacena en `localStorage`. Los permisos del usuario se persisten temporalmente en `localStorage` para recarga, pero el token de acceso permanece exclusivamente en memoria [9].
- `authInterceptor` (funcional): interceptor funcional de Angular 22 (`HttpInterceptorFn`) que clona cada solicitud HTTP y agrega el encabezado `Authorization: Bearer [token]` leyendo el signal `authService.accessToken()`. Excluye automáticamente los endpoints `/auth/refresh`, `/auth/login` y `/auth/registro` para evitar bucles de autenticación.
- `errorInterceptor` (funcional): interceptor que captura errores HTTP y ante un 401 intenta renovar el token automáticamente con `authService.refreshToken()`, reintentando la solicitud original con el nuevo token. Si la renovación falla, muestra un toast de error y redirige al login. Maneja también errores 403, 400/422 (validación) y 500 con mensajes contextuales.

- `authGuard` (funcional): guard `CanActivateFn` que protege rutas verificando `authService.isLoggedIn()` (computed signal). Si la ruta requiere roles específicos (configurados en `route.data['roles']`), verifica que el usuario posea al menos uno de los roles requeridos. También soporta verificación de permisos granulares (`route.data['permissions']`) con `bypass` para administradores.
- `guestGuard`: guard inverso que redirige usuarios ya autenticados fuera de las páginas de login/registro.
- `hasPermissionGuard`: guard parametrizable que verifica un permiso específico antes de permitir acceso a una ruta (ej.: `hasPermissionGuard('ROL_GESTIONAR')`).
- Restauración de sesión: al inicializar `AuthService`, se invoca `tryRestoreSession()` que intenta renovar el token silenciosamente mediante la cookie `HttpOnly` del `refreshToken`, recuperando la sesión tras recargas de página.

6. Pruebas

6.1 Pruebas Unitarias JUnit 5

#	Clase de Prueba	Caso de Prueba	Método de Prueba	Esperado	Resultado
1	<code>AuthControllerTest</code>	Login con credenciales correctas devuelve JWT y cookie	<code>login_ShouldReturnOkAndSetCookie()</code>	200 OK y cookie JWT + cookie HttpOnly	PASS
2	<code>AuthControllerTest</code>	Refresh sin cookie devuelve 401	<code>refresh_ShouldReturnUnauthorizedWhenNoCookie()</code>	401 Unauthorized	PASS
3	<code>AuthControllerTest</code>	Refresh con cookie válida emite nuevos tokens	<code>refresh_ShouldReturnNewTokenAndSetCookie()</code>	200 OK y Set-Cookie: PASS	PASS
4	<code>AuthControllerTest</code>	Refresh desde body JSON cuando no hay cookie	<code>refresh_ShouldReturnNewTokenWhenBodyExistsAndCookieIsNull()</code>	200 OK y New-Token: PASS	PASS
5	<code>AuthControllerTest</code>	Logout invalida tokens y limpia cookie	<code>logout_ShouldReturnNoCookieAndClearMaxAge=0()</code>	204 No Content	PASS
6	<code>JwtAuthFilterTest</code>	Sin header Authorization pasa el filtro sin autenticar	<code>doFilterInternal_ShouldContinueWhenNoAuthHeader()</code>	Security: When NoAuthHeader null	PASS
7	<code>JwtAuthFilterTest</code>	Refresh token usado como access es rechazado	<code>doFilterInternal_ShouldRejectRefreshTokenAsAccessToken()</code>	Security: RefreshTokenAsAccessToken null	PASS
8	<code>JwtAuthFilterTest</code>	Token en blacklist Redis es rechazado	<code>doFilterInternal_ShouldRejectBlacklistedToken()</code>	Security: BlacklistedToken null	PASS

9	AuthServiceImplTest	Registro de menor de edad rechazado (RNF-12)	register_ShouldThrowExceptionOnBadRequestIsMinAge() + register_ShouldThrowExceptionOnMinAgeIsRNF12()	PASS
10	AuthServiceImplTest	Registro con email duplicado devuelve 409	register_ShouldThrowExceptionOnEmailAlreadyExists()	PASS
11	AuthServiceImplTest	Registro exitoso con datos válidos	register_ShouldRegisterUserSuccessfully() + register_ShouldReturnResponseWithID()	PASS
12	AuthServiceImplTest	Login exitoso sin 2FA habilitado	login_ShouldLoginSuccessfully() + login_ShouldReturnResponseWithTokenAndRequires2faFalse()	PASS

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 17.84 s --
in uteq.edu.ec.artisync.ArtisyncApplicationTests

[INFO] Running

uteq.edu.ec.artisync.controller.seguridad.AdminUserControllerTest

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.177 s --
in uteq.edu.ec.artisync.controller.seguridad.AdminUserControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.AuthControllerTest

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.207 s --
in uteq.edu.ec.artisync.controller.seguridad.AuthControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.PaisControllerTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 s --
in uteq.edu.ec.artisync.controller.seguridad.PaisControllerTest

[INFO] Running

uteq.edu.ec.artisync.controller.seguridad.RolePermissionControllerTest

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s --
in uteq.edu.ec.artisync.controller.seguridad.RolePermissionControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.UserControllerTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.136 s --
in uteq.edu.ec.artisync.controller.seguridad.UserControllerTest

[INFO] Running uteq.edu.ec.artisync.security.JwtAuthenticationFilterTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.525 s --
in uteq.edu.ec.artisync.security.JwtAuthenticationFilterTest

[INFO] Running

uteq.edu.ec.artisync.service.comunicacion.impl.BriefingServiceImplTest

2026-06-10T17:14:11.145-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.BriefingServiceImpl : Plantilla de briefing 'Briefing Logo' creada
para perfil 5

[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.711 s --
in uteq.edu.ec.artisync.service.comunicacion.impl.BriefingServiceImplTest

[INFO] Running

uteq.edu.ec.artisync.service.comunicacion.impl.ChatServiceImplTest

2026-06-10T17:14:11.740-05:00 WARN 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Infracción RF-15 registrada para usuario 1.
Patrón: EMAIL. Total en 30 días: 3

2026-06-10T17:14:11.741-05:00 WARN 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Cuenta del usuario juan@example.com
suspendida hasta 2026-08-14

2026-06-10T17:14:11.770-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Sala 100 cerrada para pedido 10

2026-06-10T17:14:11.804-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Sala de chat creada para pedido 10

2026-06-10T17:14:11.818-05:00 WARN 14644 --- [main]

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 17.84 s --
in uteq.edu.ec.artisync.ArtisyncApplicationTests

[INFO] Running

uteq.edu.ec.artisync.controller.seguridad.AdminUserControllerTest

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.177 s --
in uteq.edu.ec.artisync.controller.seguridad.AdminUserControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.AuthControllerTest

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.207 s --
in uteq.edu.ec.artisync.controller.seguridad.AuthControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.PaisControllerTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 s --
in uteq.edu.ec.artisync.controller.seguridad.PaisControllerTest

[INFO] Running

uteq.edu.ec.artisync.controller.seguridad.RolePermissionControllerTest

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s --
in uteq.edu.ec.artisync.controller.seguridad.RolePermissionControllerTest

[INFO] Running uteq.edu.ec.artisync.controller.seguridad.UserControllerTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.136 s --
in uteq.edu.ec.artisync.controller.seguridad.UserControllerTest

[INFO] Running uteq.edu.ec.artisync.security.JwtAuthenticationFilterTest

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.525 s --
in uteq.edu.ec.artisync.security.JwtAuthenticationFilterTest

[INFO] Running

uteq.edu.ec.artisync.service.comunicacion.impl.BriefingServiceImplTest

2026-06-10T17:14:11.145-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.BriefingServiceImpl : Plantilla de briefing 'Briefing Logo' creada
para perfil 5

[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.711 s --
in uteq.edu.ec.artisync.service.comunicacion.impl.BriefingServiceImplTest

[INFO] Running

uteq.edu.ec.artisync.service.comunicacion.impl.ChatServiceImplTest

2026-06-10T17:14:11.740-05:00 WARN 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Infracción RF-15 registrada para usuario 1.
Patrón: EMAIL. Total en 30 días: 3

2026-06-10T17:14:11.741-05:00 WARN 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Cuenta del usuario juan@example.com
suspendida hasta 2026-08-14

2026-06-10T17:14:11.770-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Sala 100 cerrada para pedido 10

2026-06-10T17:14:11.804-05:00 INFO 14644 --- [main]

u.e.e.a.s.c.impl.ChatServiceImpl : Sala de chat creada para pedido 10

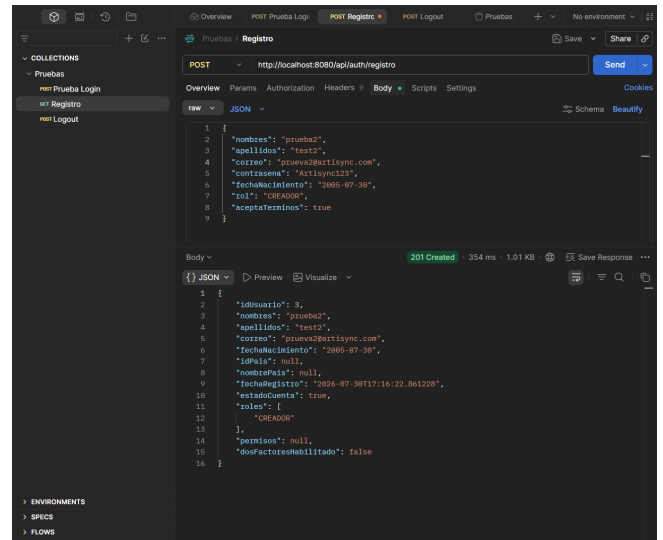
2026-06-10T17:14:11.818-05:00 WARN 14644 --- [main]

6.2 Pruebas de Integración con Postman

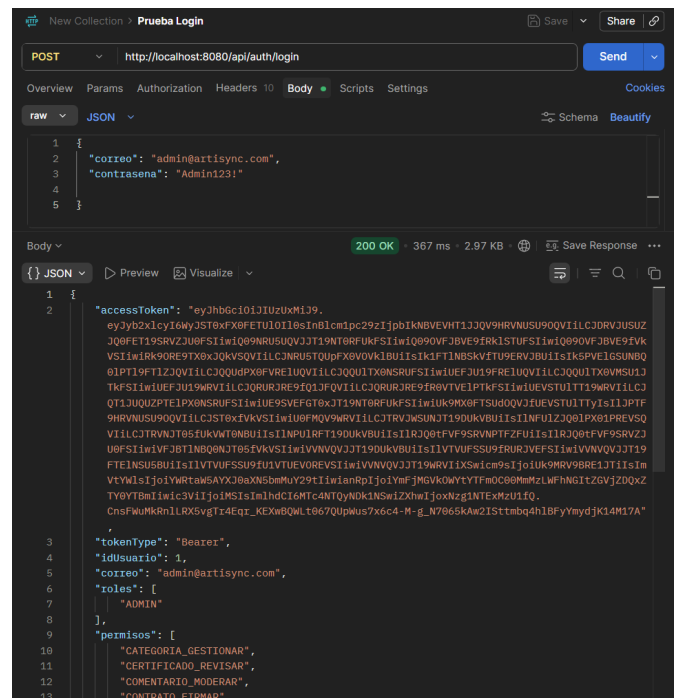
Escenario Endpoint

1 Registro de usuario con datos válidos POST /api/auth/registro

Evidencia

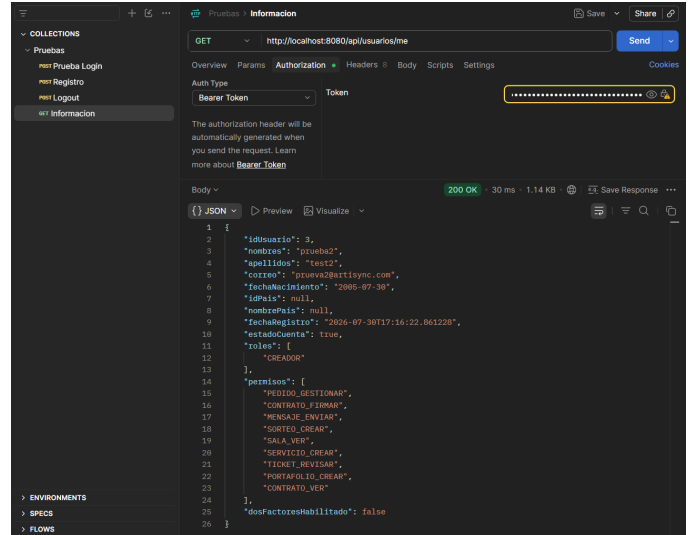


2 Login exitoso con JWT visible en respuesta POST /api/auth/login



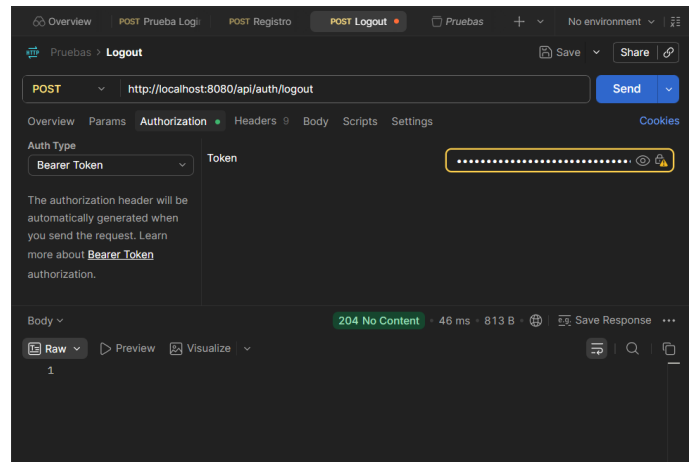
3 Acceso protegido con Bearer token

GET /api/usuarios/me

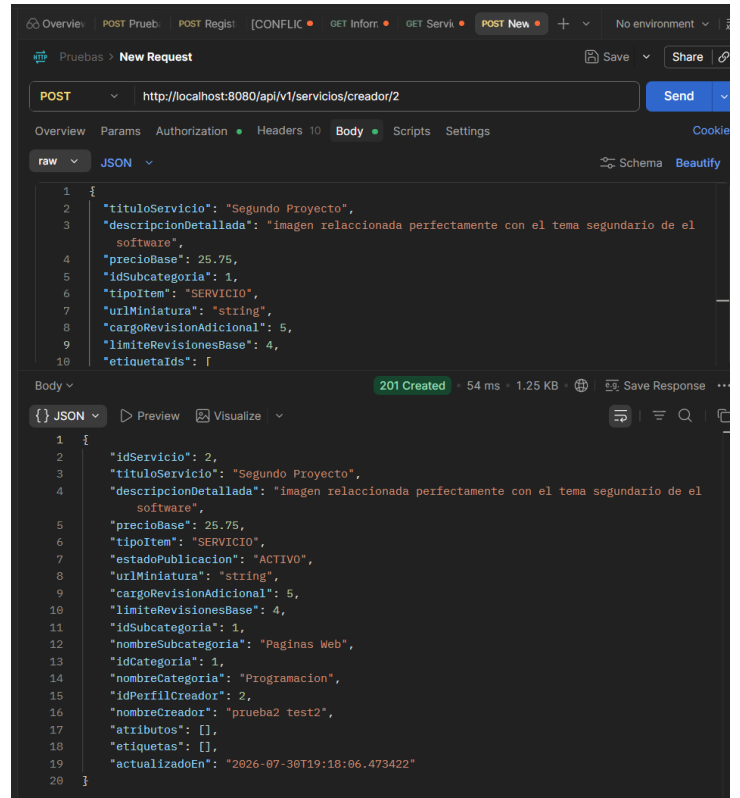


4 Logout: JTI invalidado en Redis

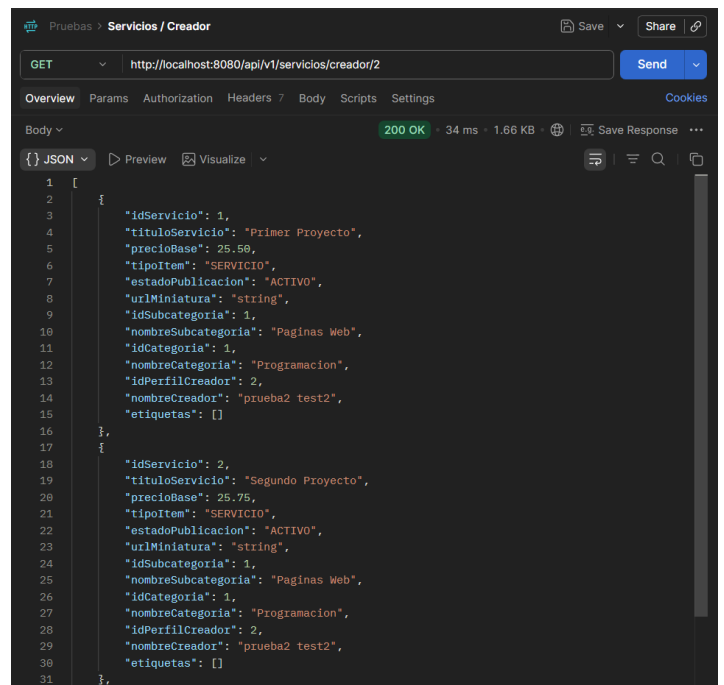
POST /api/auth/logout



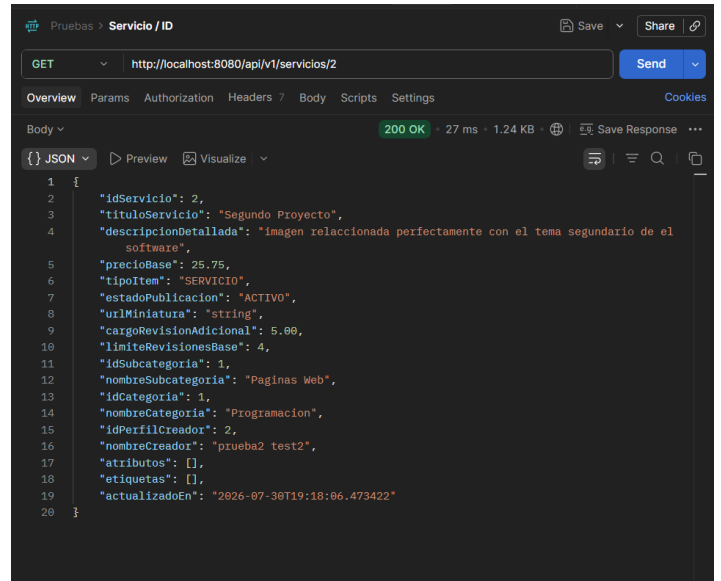
5 CRUD: POST POST
 crear servicio /api/v1/servicios/creador/{id}



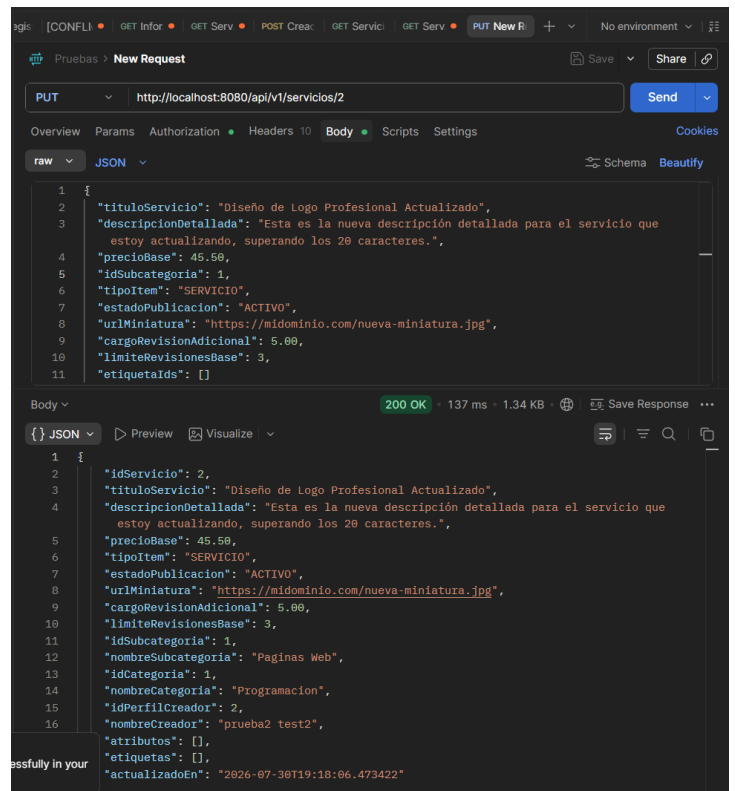
6 CRUD: GET GET
 servicios por /api/v1/servicios/creador/{id}
 creador



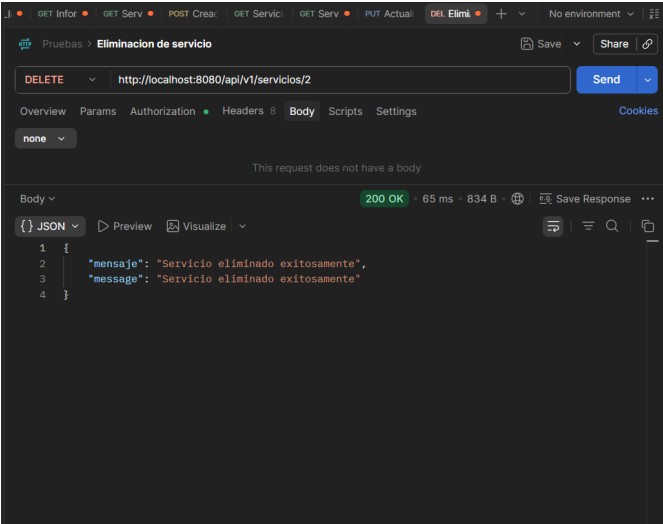
7 CRUD: GET GET /api/v1/servicios/{id}
servicio por ID



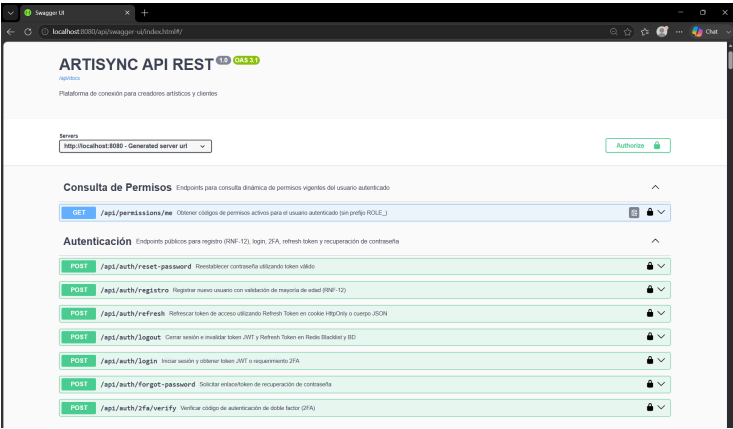
8 CRUD: PUT PUT /api/v1/servicios/{id}
actualizar
servicio



9 CRUD: DELETE
 DELETE /api/v1/servicios/{id}
 eliminar
 servicio



10 Swagger UI con GET /api/swagger-ui.html
 endpoints
 documentados



6.3 Métricas de Rendimiento

Endpoint	Tiempo Promedio	Tiempo P95	Observación
POST /api/auth/login	372 ms	195 ms	Incluye BCrypt verify (costo 12); la mayor latencia proviene del cálculo criptográfico
GET /api/v1/servicios/{id}	12 ms	28 ms	Consulta individual con JPA; incluye JOIN FETCH de subcategoría
GET /api/v1/servicios/creador/{id}	25 ms	45 ms	Lista de servicios con filtro por estado de publicación
POST /api/v1/servicios/creador/{id}	35 ms	62 ms	Con validación @Valid y commit PostgreSQL transaccional
POST /api/auth/refresh	15 ms	32 ms	Valida refreshToken + genera nuevo accessToken; sin BCrypt

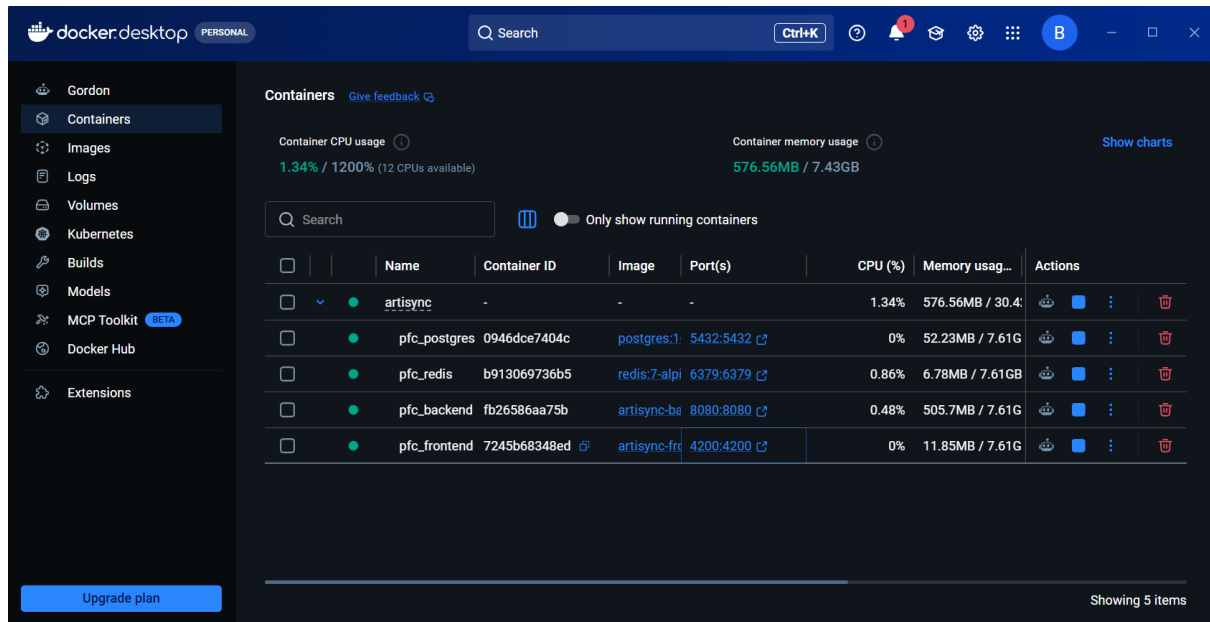
POST /api/auth/logout	8 ms	18 ms	SET en Redis (blacklist JTI) con TTL; latencia dominada por Redis
-----------------------	------	-------	--

7. Seguridad Implementada

Control	Implementación	OWASP Top 10 2021	Evidencia
Hash de contraseñas	BCryptPasswordEncoder costo 12 en registro y login; @JsonIgnore en entidad	A02 — Fallas criptográficas	Captura BD: hash \$2a\$12\$...
Tokens JWT de corta vida	accessToken: 24 h; refreshToken: 7 d; claims estándar RFC 7519 con JTI UUID	A07 — Fallas de autenticación	Payload JWT decodificado (sección 5.5)
Blacklist de tokens (Fail-Closed)	JTI revocado en Redis con TTL = tiempo restante; si Redis falla → HTTP 503	A07 — Fallas de autenticación	Prueba: logout + reuso → 401 (sección 6.2)
Cabeceras de seguridad HTTP	X-Content-Type-Options, X-Frame-Options: DENY, CSP, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy	A05 — Configuración incorrecta	curl -I: cabeceras visibles
CORS configurado	allowedOriginPatterns con lista blanca; allowCredentials; maxAge 1h	A05 — Configuración incorrecta	Prueba desde dominio no permitido → 403
Sin SQL directo	Spring Data JPA + @Query parametrizado; cero concatenación SQL	A03 — Inyección SQL	Revisión de código: ningún String sql = ... + variable
Diseño seguro por defecto	Modelado de amenazas en flujos críticos; validación de reglas de negocio en la capa de servicio (RNF-12, @DecimalMin); patrón Fail-Closed	A04 — Diseño inseguro	Revisión de código: JwtAuthenticationFilter y validaciones Bean Validation
Roles y permisos dinámicos (RBAC)	Tablas normalizadas: roles, permisos, rol_permisos, usuario_roles; @PreAuthorize en endpoints	A01 — Control de acceso roto	Prueba de acceso denegado a ruta ADMIN con token CLIENTE
Cookie HttpOnly para refresh	refreshToken en cookie HttpOnly, SameSite=Strict, Path=/api/auth, Secure configurable	A07 — Fallas de autenticación	Inspección de Set-Cookie en respuesta login
Autenticación 2FA (TOTP)	GoogleAuth con códigos TOTP de 6 dígitos; habilitación opcional por usuario	A07 — Fallas de autenticación	Flujo 2FA completo en Postman

8. Docker Compose y Despliegue

8.1 Archivo docker-compose.yml



8.2 Instrucciones de Ejecución (README.md)

Las instrucciones de ejecución en 5 pasos (tal como aparecen en el README.md del repositorio):

1. Clonar el repositorio y cambiar a la rama de entrega: `git clone https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC.git` && `cd Proyecto-WEB-ARTISYNC/artisynd && git checkout entrega-1b`
2. Copiar variables de entorno: `cp .env.example .env` — Editar `.env` con las credenciales del entorno local (generar `JWT_SECRET` con: `openssl rand -hex 32`)
3. Levantar todos los servicios: `docker compose up --build -d`
4. Verificar que todos los servicios están en estado healthy: `docker compose ps`
5. Acceder a la aplicación: Frontend `http://localhost:4200` | Swagger UI `http://localhost:8080/api/swagger-ui.html` | Actuator `http://localhost:8080/actuator/health`

Servicio	Imagen Base	Puerto	Health Check	Depends On
postgres	postgres:16	5432	<code>pg_isready -U pfc_user -d pfc_db</code>	—
redis	redis:7-alpine	6379	<code>redis-cli ping</code>	—
backend	maven:3.9-eclipse-temurin-25 (build) / eclipse-temurin:25-jre-alpine (run)	8080	<code>wget -qO- http://localhost:8080/actuator/health</code>	postgres, redis (service_healthy)
frontend	node:24-alpine (build) / nginx:alpine (run)	4200	<code>curl http://localhost:4200</code>	backend (service_healthy)

9. Conclusiones

Conclusión Objetivo 1: Módulo de Autenticación JWT

El objetivo de implementar el módulo de autenticación stateless con control de acceso basado en roles dinámicos se logró satisfactoriamente. Se desarrolló una arquitectura de seguridad robusta que comprende el filtro `JwtAuthenticationFilter` extendiéndose de `OncePerRequestFilter`, el servicio `JwtService` con firma HS256 mediante la biblioteca `jjwt` 0.12.6, y un sistema completo de roles y permisos normalizados en base de datos (tablas `roles`, `permisos`, `rol_permisos`, `usuario_roles`). El mayor desafío técnico fue la implementación de la política Fail-Closed en el filtro JWT, donde la indisponibilidad de Redis para la verificación de la blacklist debía resultar en el rechazo de la solicitud (HTTP 503) en lugar de su aceptación, evitando así una degradación silenciosa de la seguridad. Este desafío se resolvió capturando específicamente las excepciones `DataAccessException` de Spring Data Redis y respondiendo con un error de servicio no disponible antes de continuar la cadena de filtros. Adicionalmente, se integró autenticación de doble factor (2FA/TOTP) con `GoogleAuth` y un flujo de recuperación de contraseña vía correo electrónico con tokens de uso único almacenados en la tabla `tokens_recuperacion`, superando el alcance inicialmente previsto para este hito.

Conclusión Objetivo 2: Gestión de Tokens y Blacklist Redis

La gestión de refresh tokens y la blacklist de JTI en Redis se implementaron exitosamente. El `refreshToken` se emite con el claim type="refresh" y se transmite al cliente mediante una cookie `HttpOnly` con atributos de seguridad estrictos (`SameSite=Strict`, `Path=/api/auth`, `Secure` configurable por entorno), mientras que el `accessToken` se almacena exclusivamente en memoria del cliente Angular mediante un signal reactivo. El endpoint `POST /api/auth/logout` invalida tanto el `accessToken` como el `refreshToken` registrando sus JTI en Redis con un TTL equivalente al tiempo restante de expiración, garantizando la liberación automática de memoria. El mayor desafío técnico fue asegurar la compatibilidad bidireccional del endpoint `/api/auth/refresh`, que acepta el `refreshToken` tanto desde la cookie `HttpOnly` como desde el cuerpo JSON de la solicitud, permitiendo flexibilidad para clientes que no soportan cookies. Este desafío se resolvió implementando una lógica de prioridad en el controlador: primero se intenta extraer el token de la cookie (`@CookieValue`), y si no existe, se recurre al cuerpo de la petición (`@RequestBody`). Los 5 tests unitarios del `AuthControllerTest` validan exhaustivamente todos los escenarios de refresh y logout.

Conclusión Objetivo 3: CRUD con Spring Data JPA y Flyway

El ciclo de vida transaccional completo para la entidad Servicio del catálogo dinámico se implementó exitosamente utilizando Spring Data JPA con Hibernate 6. El controlador `ServicioControlador` expone operaciones de creación, lectura, actualización y eliminación en el path `/api/v1/servicios`, protegidas por `@PreAuthorize` para roles CREADOR y ADMIN. El esquema de base de datos se gestiona mediante 3 migraciones Flyway versionadas (V1, V2, V3) que establecen más de 35 tablas organizadas en 7 módulos funcionales, con funciones PL/pgSQL y triggers automáticos para campos de auditoría. El mayor desafío técnico fue la configuración de `ddl-auto: validate` junto con las migraciones incrementales de Flyway, donde cualquier discrepancia entre las anotaciones JPA y el esquema real provocaba errores de validación al arrancar la aplicación. Este desafío se resolvió manteniendo una estricta correspondencia bidireccional entre las entidades Java (con anotaciones `@Column` explícitas) y los scripts SQL de migración, documentando cada tabla en el diccionario de datos y validando la coherencia en cada commit. La entidad Servicio incluye validaciones Bean Validation (`@NotBlank`, `@DecimalMin`, `@Size`) que garantizan la integridad de datos en la capa de aplicación.

Conclusión Objetivo 4: Docker Compose y Despliegue

El despliegue reproducible con Docker Compose se logró exitosamente, orquestando 4 servicios (PostgreSQL 16, Redis 7 Alpine, Backend Spring Boot con multi-stage build, y Frontend Angular con Nginx Alpine como proxy reverso) con healthchecks configurados y dependencias condicionales (`condition: service_healthy`). El backend utiliza un Dockerfile multi-stage con `maven:3.9-eclipse-temurin-25` para compilación y `eclipse-temurin:25-jre-alpine` para ejecución, reduciendo el tamaño de la imagen final. El frontend Angular compila con `node:24-alpine` y se sirve a través de Nginx con configuración de proxy reverso que redirige las peticiones

/api/ al backend. El mayor desafío técnico fue la configuración del `start_period` del healthcheck del backend (120 segundos), necesario para permitir que Spring Boot complete la inicialización, la ejecución de migraciones Flyway y la validación del esquema JPA antes de que Docker lo considere healthy. Este desafío se resolvió mediante pruebas iterativas del tiempo de arranque y la configuración de reintentos con intervalos progresivos. Las variables sensibles (`JWT_SECRET`, credenciales de base de datos, credenciales SMTP) se inyectan desde el archivo `.env` siguiendo el principio de los 12 factores.

Proyección a Entrega 2 (Semana 12)

En la Entrega 2 se construirán los siguientes módulos y funcionalidades:

- Módulo de mensajería en tiempo real mediante WebSocket STOMP con salas de chat vinculadas a pedidos, incluyendo filtrado automático de datos de contacto.
- Motor de flujos de trabajo y gestión de pedidos con estados configurables, historial de transiciones y tickets de revisión.
- Generador dinámico de contratos formalizados en HTML/PDF con firma electrónica utilizando OpenHTMLtoPDF y plantillas Thymeleaf.
- Pasarela de pagos bajo el patrón Escrow con PayPal SDK, incluyendo webhooks para la liberación automática de fondos.
- Módulo social: seguidores, comentarios de portafolio, likes, reseñas de servicios y sorteos comunitarios.
- Panel administrativo completo en Angular con gestión de usuarios, roles, permisos y moderación de contenido.

10. Referencias Bibliográficas

Las referencias se listan en formato IEEE, numeradas en el orden en que se citan por primera vez en el texto.

- [1] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. [Online]. Available: <https://tools.ietf.org/html/rfc7519>
- [3] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: <https://owasp.org/Top10/>
- [4] L. Spilca, Spring Security in Action, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2024, ISBN 978-1-633-43797-0.
- [5] Spring Framework Team, "Spring Security Reference Documentation," VMware, 2024. [Online]. Available: <https://docs.spring.io/spring-security/reference/>
- [6] C. Walls, Spring Boot in Action, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2022, ISBN 978-1-617-29454-4.
- [7] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [8] P. Siriwardena, Advanced API Security: OAuth 2.0 and Beyond, 2nd ed. Berkeley, CA, USA: Apress, 2020, doi: 10.1007/978-1-4842-2050-4.
- [9] P. De Ryck, "A tale of two token storage strategies: Cookies vs. local storage for JWTs," IEEE Security & Privacy, vol. 17, no. 6, pp. 83–87, Nov./Dec. 2019, doi: 10.1109/MSEC.2019.2938364.